

第 6 节：对任意 $r \geq 2$ 的 $(O(rN^{1/r}), O(N^{1-1/r}))$ 实现

Robert E. Tarjan Uri Zwick
中文翻译（按原文段落对应）

6 对任意 $r \geq 2$ 的 $(O(rN^{1/r}), O(N^{1-1/r}))$ 实现

推广第 5 节的构造，我们对每个 $r \geq 2$ 得到一个 $(O(rN^{1/r}), O(N^{1-1/r}))$ 实现。也可以像第 5 节开头那样，以递归方式构造这些实现。本节给出它们的非递归版本。

对于给定的 $r \geq 2$ ，数据结构维护参数 B ，使得

$$N^{1/r} \leq B < 4N^{1/r}.$$

它把元素保存在大小为 $B, B^2, \dots, B^{r-1}$ 的块中。

若只需支持增长而无需支持收缩，我们可以维持如下不变式：每一种大小的块始终至多有 B 个。为了高效支持收缩操作，我们稍后放宽这一条件，允许每一种大小至多有 $2B$ 个块。大小为 $B^2, \dots, B^{r-1}$ 的块始终是满的。所有长度为 B 的块也都是满的，只有最后一个块可能例外。若最后一个大小为 B 的块尚未填满，则增长操作很简单。若它已满，并且大小为 B 的块少于 B 个，就分配一个新的大小为 B 的块，并把新元素放入其中。若某个大小 B^i 已有 B 个满块，就分配一个大小为 B^{i+1} 的新块，按正确次序把这 B 个块中的元素复制到新块中，再释放这 B 个旧块。随后再分配一个大小为 B 的新块，并把新元素放入其中。这个过程模拟了一个 B 进制计数器的进位操作。

当 $N = B^r$ 时，将 B 加倍并重建整个数据结构。也可以把重建推迟到数据结构完全填满时，即

$$N = B^r + B^{r-1} + \dots + 1 = \frac{B^{r+1} - 1}{B - 1},$$

但这样做获得的收益并不大。为简化叙述，我们在 $N = B^r$ 时重建。

数据结构维护 $r - 1$ 个索引块，每一种可能的数据块大小对应一个索引块。存储一个包含 N 个元素的数组时，所用总空间至多为

$$N + (2r - 1) + 2(r - 1)B + B = N + O(rN^{1/r}).$$

实现所需的最大额外空间为 $O(N^{1-1/r})$ ，它出现在分配一个大小为 B^{r-1} 的新块时。

很容易看出，每个元素都能在 $O(r)$ 最坏情形时间内访问。稍作处理后，这一界可以降为 $O(1)$ 最坏情形时间，见第 6.3 节。

增长操作的摊还代价为 $O(r)$ ，因为每个元素最初被放入大小为 B 的块，随后至多依次移动到大小为 B^i 的块，其中 $i = 2, \dots, r - 1$ 。完整重建只会给一次增长操作的摊还代价增加一个常数，因为两次重建之间，数据结构中的元素总数会增加一个常数倍。

在保持每次增长和收缩操作摊还代价为 $O(r)$ 的同时支持收缩并不困难。一个标准方法是使用冗余的 B 进制计数器。对于每一种大小，我们允许至多 $2B$ 个块，而不只允许 B 个。当某个大小为 B^i 的层级有 $2B$ 个满块时 ($1 \leq i < r-1$)，分配一个大小为 B^{i+1} 的新块，把前 B 个大小为 B^i 的块复制到其中，再释放这些旧块。若执行收缩时不存在大小为 B 的块，则选择最小的、存在大小为 B^i 块的 i ，把其中一个块拆分为：每种大小 $B^{i-1}, \dots, B^2$ 各 $B-1$ 个块，以及 B 个大小为 B 的块。第 6.2 节将证明，增长和收缩操作的摊还代价仍为 $O(r)$ 。Kaplan 等人 [10] 在另一个问题中使用过类似的冗余 B 进制计数器。

图 4 (结构示意图) 对任意 $r \geq 2$ 的 $(O(rN^{1/r}), O(N^{1-1/r}))$ 实现。

顶层保存 $N, B, n_0, n_1, \dots, n_{r-1}$ 以及指向各索引块的指针；第 i 个索引块至多指向 $2B$ 个大小为 B^i 的数据块。

6.1 实现细节

增长和收缩操作的伪代码见图 5。与图 4 一样，设 n_i ($i \in [r-1]$) 为大小为 B^i 的数据块数量，并把这些块记作

$$A[i][0], \dots, A[i][n_i - 1].$$

若 $n_1 > 0$ ，令 n_0 表示最后一个大小为 B 的数据块 $A[1][n_1 - 1]$ 中的元素数。采用这一记号时，对 $1 \leq i \leq r-1$ ， $A[i]$ 是第 i 个索引块，其大小为 $2B$ ； A 是一个大小为 r 的块，保存指向这些索引块的指针。每个块的下标都从 0 开始。为方便起见，把单元 $A[0]$ 留空。数值 $n_0, n_1, \dots, n_{r-1}$ 保存在一个大小为 r 的块 n 中。为简化记号，我们写 n_i 而不写 $n[i]$ 。块 A 和 $A[i]$ ($1 \leq i \leq r-1$) 只在重建操作期间分配或释放。

Listing 1: 图 5: $(O(rN^{1/r}), O(N^{1-1/r}))$ 实现的增长与收缩伪代码 (中文注释版)

```
Grow(a):
  if N = B^r:
    Rebuild(2B)
  else if n_1 = 2B and n_0 = B:
    Combine-Blocks()
  else if n_1 = 0 or n_0 = B:
    A[1][n_1] <- Allocate(B)
    n_1 <- n_1 + 1
    n_0 <- 0
  A[1][n_1 - 1][n_0] <- a
  n_0 <- n_0 + 1
  N <- N + 1

Combine-Blocks():
  k <- min { i in [r-1] | n_i < 2B }
  if k = infinity: error
  for i <- k-1 downto 1:
    A[i+1][n_{i+1}] <- Allocate(B^{i+1})
    for j <- 0 to B-1:
      Copy(A[i][j], 0,
```

```

        A[i+1][n_{i+1}], j B^i, B^i)
    Deallocate(A[i][j])
    A[i][j] <- A[i][j+B]    // 平移指针下标
    n_i <- B
    n_{i+1} <- n_{i+1} + 1

Shrink():
    if N = (B/4)^r:
        Rebuild(B/2)
    else if n_1 = 0:
        Split-Blocks()
    n_0 <- n_0 - 1
    N <- N - 1
    if n_0 = 0:
        Deallocate(A[1][n_1 - 1])
        n_0 <- B
        n_1 <- n_1 - 1

Split-Blocks():
    k <- min { i in [r-1] | n_i > 0 }
    if k = infinity: error
    for i <- k-1 downto 1:
        n_{i+1} <- n_{i+1} - 1
        for j <- 0 to B-1:
            A[i][j] <- Allocate(B^i)
            Copy(A[i+1][n_{i+1}], j B^i,
                A[i][j], 0, B^i)
        Deallocate(A[i+1][n_{i+1}])

```

伪代码假设存在过程 $\text{Allocate}(\ell)$ ，它分配一个大小为 ℓ 的新块，并返回指向新块的指针；还假设存在过程 $\text{Deallocate}(X)$ ，用于释放先前分配的块 X 。此外， $\text{Copy}(X, x, Y, y, \ell)$ 把 $X[x..x + \ell - 1]$ 中的元素复制到 $Y[y..y + \ell - 1]$ 。这里， $X[x..x + \ell - 1]$ 是 $X[x], X[x + 1], \dots, X[x + \ell - 1]$ 的简写。也就是说，它执行赋值

$$Y[y + i] \leftarrow X[x + i], \quad i = 0, 1, \dots, \ell - 1.$$

伪代码中未给出的过程 $\text{Rebuild}(B')$ 使用新参数 B' 重建数据结构；通常 B' 取 $2B$ 或 $B/2$ 。

$\text{Grow}(a)$ 的工作方式如下。若 $N = B^r$ ，就令 $B \leftarrow 2B$ 并重建数据结构。若第一层完全填满，即 $n_1 = 2B$ 且 $n_0 = B$ ，则调用 Combine-Blocks 。该过程寻找满足 $n_k < 2B$ 的最小下标 $k \in [r - 1]$ ；这样的下标必然存在。随后对 $i = k - 1, k - 2, \dots, 1$ ，它把 B 个大小为 B^i 的块合并为一个大小为 B^{i+1} 的新块，并按需要分配和释放块。

更具体地说，为保持元素次序，把前 B 个大小为 B^i 的块

$$A[i][0], \dots, A[i][B - 1]$$

合并为新的最后一个大小为 B^{i+1} 的块 $A[i + 1][n_{i+1}]$ 。原来的

$$A[i][B], \dots, A[i][2B - 1]$$

变成新的

$$A[i][0], \dots, A[i][B-1].$$

这一步只涉及指针修改，不移动元素。把 $A[i]$ 视为循环数组即可避免这些指针修改。最后，把大小为 B^i 的块数设为 B ，并把大小为 B^{i+1} 的块数增加 1，即令 $n_i \leftarrow B$ 、 $n_{i+1} \leftarrow n_{i+1} + 1$ 。

若既未调用 Rebuild，也未调用 Combine-Blocks，则 Grow(a) 检查大小为 B 的块中是否有空位。若没有空位，即 $n_1 = 0$ （不存在大小为 B 的块），或者 $n_1 > 0$ 且 $n_0 = B$ （所有大小为 B 的块都已满），就分配一个新的空块 $A[1][n_1]$ ，把 n_1 加 1，并把 n_0 设为 0。最后，在所有情况下，新元素 a 都被放入空块或部分填满块中的第一个空位，即

$$A[1][n_1 - 1][n_0] \leftarrow a,$$

随后把 n_0 和 N 各加 1。

Shrink 与此类似。若 $N = (B/4)^r$ ，则令 $B \leftarrow B/2$ 并重建数据结构。否则，若 $n_1 = 0$ ，即不存在大小为 B 的块，Split-Blocks 就寻找满足 $n_i > 0$ 的最小 $k \in [r-1]$ 。最后一个大小为 B^k 的块被拆分为：每一种大小 $B^{k-1}, \dots, B^2$ 各 $B-1$ 个块，以及 B 个大小为 B 的块。最后，在所有情况下，都丢弃最后一个大小为 B 的块中的最后一个元素，并把 n_0 和 N 各减 1。若 $n_0 = 0$ ，就释放这个空的大小为 B 的块，把 n_1 减 1，并把 n_0 设为 B 。实际实现中，最好像第 3.2 节的 $r = 2$ 情形和第 5 节的 $r = 3$ 情形那样，延迟释放最后一个大小为 B 的块；相应修改伪代码并不困难。

定理 6.1. 该实现是可变长数组的一个 $(O(rN^{1/r}), O(N^{1-1/r}))$ 实现。增长和收缩操作的摊还代价为 $O(r)$ 。访问操作可以在 $O(1)$ 最坏情形时间内支持。

证明。实现的正确性，以及 $O(rN^{1/r})$ 和 $O(N^{1-1/r})$ 的空间界，都由前面的讨论直接得到。增长和收缩操作摊还代价为 $O(r)$ 的结论将在下面的引理 6.3 中证明。把访问操作实现为 $O(1)$ 最坏情形时间的方法见下面引理 6.4 的证明。□

令 $r = \log N$ ，可得到下面这个有趣的推论。

推论 6.2. 存在一种可变长数组实现，存储大小为 N 的数组时使用 $N + O(\log N)$ 空间；增长和收缩操作的摊还时间为 $O(\log N)$ ；访问操作的最坏情形时间为 $O(1)$ 。

6.2 增长和收缩操作的摊还代价

引理 6.3. 在上述 $(O(rN^{1/r}), O(N^{1-1/r}))$ 实现中，增长和收缩操作的摊还代价为 $O(r)$ 。

证明。我们把一次 Grow 或 Shrink 操作的代价定义为它执行的元素赋值次数，并证明这些代价的摊还界为 $O(r)$ 。一次 Grow 或 Shrink 执行的全部基本操作数，与这里定义的代价加 1 成正比，因此这足以证明引理。加 1 是为了处理不执行任何元素赋值的 Shrink 操作。

一次 Grow 的代价为 1，再加上它调用 Combine-Blocks 时后者的代价。类似地，一次 Shrink 的代价为 0，再加上它调用 Split-Blocks 时后者的代价。大多数 Shrink 只减小 N 和 n_0 ，不执行元素赋值。若一次 Combine-Blocks 生成大小为 B^k 的块，则其代价为

$$B(B + B^2 + \dots + B^{k-1}) \leq 2B^k,$$

其中假设 $B \geq 2$ 。若一次 Split-Blocks 拆分一个大小为 B^k 的块，则其代价为 B^k 。

当前位于第 i 层、即位于大小为 B^i 的数据块中的每个元素，被赋予 $r - i$ 个信用单位。每个信用单位可以支付一次元素赋值。一次 Grow 的代价（不包括可能调用的 Combine-Blocks）为 1。新元素总是被放在第 1 层，因此给它分配的信用为 $r - 1$ ；所以不包括 Combine-Blocks 的一次 Grow 的摊还代价为 r 。

一次 Combine-Blocks 的摊还代价为 0，因为每次元素赋值都把一个元素从第 i 层移到第 $i + 1$ 层，使该元素的信用减少 1，正好支付这次赋值。因此，Grow 的摊还代价至多为 r 。

为处理 Shrink，我们还给各层分配信用。每次 Shrink 都给每一层增加 3 个信用单位。因此，不包括 Split-Blocks 的一次 Shrink 的摊还代价至多为 $3r$ 。

下面证明一次 Split-Blocks 的摊还代价至多为 0。先注意，两次连续拆分大小为 B^k 的块之间，至少执行了 B^k 次 Shrink。原因是：一次这样的拆分之后，第 1 层到第 $k - 1$ 层恰好包含 B^k 个元素；一次生成新的大小为 B^k 块的 Combine-Blocks 之后，第 1 层到第 $k - 1$ 层也至少包含 B^k 个元素。因此，在第 k 层的一次 Split-Blocks 与此前第 k 层最近一次 Combine-Blocks 或 Split-Blocks 之间，至少发生了 B^k 次 Shrink。同理，在第 k 层第一次执行 Split-Blocks 之前，也至少发生了 B^k 次 Shrink。所以，当第 k 层发生 Split-Blocks 时，该层已经积累至少 $3B^k$ 个未使用的信用单位。

第 k 层一次 Split-Blocks 的实际代价为 B^k 。把元素从第 k 层移到更低层之后，需要额外补充给这些元素的信用总量为

$$B(k-1) + (B-1) \sum_{i=1}^{k-1} (k-i)B^i \leq B \sum_{i=1}^{k-1} (k-i)B^i \leq B^{k+1} \sum_{j \geq 1} jB^{-j} = \frac{B^{k+2}}{(B-1)^2} \leq 2B^k,$$

最后一个不等式假设 $B \geq 4$ 。因此，第 k 层的 $3B^k$ 个信用单位足以支付 Split-Blocks 的实际代价和补充信用所需的代价，其摊还代价至多为 0。

最后还需计入重建的代价。现在让每次 Grow 或 Shrink 再给整个数据结构增加 $2r$ 个信用单位；这会把增长和收缩的摊还代价都增加 $2r$ 。一次把 B 加倍的 Rebuild 与上一次 Rebuild 之间，至少发生

$$B^r - (B/2)^r = N(1 - 2^{-r}) \geq N/2$$

次 Grow。一次把 B 减半的 Rebuild 与上一次重建之间，至少发生

$$(B/2)^r - (B/4)^r = (2^r - 1)N \geq N$$

次 Shrink。所以在即将重建时，数据结构至少积累了 rN 个未使用信用单位。一次 Rebuild 的实际代价恰为 N ，而重新给所有元素分配信用所需的总量至多为 $(r-1)N$ 。因此，Rebuild 的摊还代价为 0。

上面对 Rebuild 的摊还分析十分宽松。当一次 Rebuild 把 B 加倍时，所有元素的总信用实际上会减少。因此，每次 Grow 只需给数据结构增加

$$\frac{1}{1 - 2^{-r}} \leq 2$$

个信用单位。类似地，每次 Shrink 只需增加

$$\frac{r}{2^r - 1} \leq 1$$

个信用单位。这不会改变增长和收缩操作的 $O(r)$ 摊还代价。

上述分析隐含地假设 r 是常数，即 r 不随 N 变化。把分析调整到非常数 $r = r(N)$ 的情形并不困难。 $\square$

6.3 在 $O(1)$ 最坏情形时间内访问和修改元素

本节说明, 第 6.1 节的 $(O(rN^{1/r}), O(N^{1-1/r}))$ 实现可以在 $O(1)$ 最坏情形时间内支持访问和修改操作。我们假设可以在 $O(1)$ 时间内求出一个机器字 x 中最高有效置位的下标, 记作 $\text{msb}(x)$ 。后文会进一步讨论这一假设。

引理 6.4. 上述 $(O(rN^{1/r}), O(N^{1-1/r}))$ 实现可以在 $O(1)$ 最坏情形时间内支持访问和修改操作。

为简化访问操作的高效实现, 我们对数据结构作一个小改动。令 n_1 表示满的大小为 B 的数据块数量, 不把可能存在的、含有 $n_0 < B$ 个元素的部分填满块计入其中。 $n_2, \dots, n_{r-1}$ 仍表示大小为 $B^2, \dots, B^{r-1}$ 的满块数量。还假设 B 是 2 的幂, 即对某个整数 $b > 0$ 有 $B = 2^b$ 。

注意

$$N = \sum_{j=0}^{r-1} n_j B^j = (n_{r-1}, \dots, n_1, n_0)_B,$$

其中对 $0 \leq j < r$ 有 $0 \leq n_j \leq 2B$ 。因此, $(n_{r-1}, \dots, n_1, n_0)$ 是 N 的一个可能冗余的 B 进制表示。把它紧凑地存入两个机器字

$$N_0 = (n_{r-1}^0, \dots, n_1^0, n_0^0)_B, \quad N_1 = (n_{r-1}^1, \dots, n_1^1, n_0^1)_B,$$

其中

$$n_j = n_j^0 + B n_j^1, \quad 0 \leq n_j^0 < B, \quad 0 \leq n_j^1 \leq 2.$$

对 $a = 0, 1$, N_a 是各个 n_j^a 的 b 位二进制表示的拼接。利用简单的逻辑和移位操作, 可以在常数时间内从 N_a 中提取任意 n_j^a 。数据结构可以在维护 N 的同时轻松维护 N_0 和 N_1 。

定义

$$N_k = \sum_{j=0}^{k-1} n_j B^j, \quad 0 \leq k < r.$$

其中 $N_r = N$ 、 $N_0 = 0$ 。 N_k 是大小至多为 B^{k-1} 的块中所含元素总数, 因此 $N - N_k$ 是存放在大小至少为 B^k 的块中的元素数。回忆一下, 大块保存下标较小的元素。因此, 只有当

$$N - N_{k+1} \leq i$$

时, 第 i 个元素才会位于大小至多为 B^k 的块中。数值 N_k 可以由数据结构显式维护, 而不会影响各操作的渐近运行时间。也可以令

$$N_k = (N_0 \& M_k) + B(N_1 \& M_k),$$

其中 $\&$ 表示按位与, M_k 是由 kb 个 1 组成的掩码, 即

$$M_k = 2^{kb} - 1 = (1 \ll kb) - 1.$$

这些掩码可以预先计算。乘以 B 也能用左移 b 位替代。

要访问可变长数组中的第 i 个元素 ($0 \leq i < N$), 需要找到唯一的 $0 \leq k < r$, 使

$$N - N_{k+1} \leq i < N - N_k.$$

第 i 个元素位于某个大小为 B^k 的块中：其块内位置为

$$(i - (N - N_{k+1})) \bmod B^k,$$

块编号为

$$\left\lfloor \frac{i - (N - N_{k+1})}{B^k} \right\rfloor.$$

若 $k = 0$, 第 i 个元素就是唯一部分填满的大小为 B 的块中的第 $i - (N - N_1)$ 个元素。由于 B^k 是 2 的幂, 一旦知道 k , 就能用逻辑和移位操作在常数时间内访问第 i 个元素。具体地, $x \bmod B^k = x \& M_k$, 且 $\lfloor x/B^k \rfloor = x \gg bk$ 。

令

$$x = (N - 1) - i,$$

即从数组末端开始、仍从 0 编号时, 第 i 个元素的下标。此时需要找到唯一的 $0 \leq k < r$, 使

$$N_k \leq x < N_{k+1}.$$

把 x 的标准 B 进制表示写作

$$x = (x_{r-1}, \dots, x_1, x_0)_B.$$

为求出 k , 先找满足 $x_\ell > 0$ 的最大下标 ℓ 。这可通过求 x 的最高有效置位下标实现:

$$\ell \leftarrow \left\lfloor \frac{\text{msb}(x)}{b} \right\rfloor.$$

由于 $x_\ell > 0$, 有 $x \geq B^\ell$ 。另一方面, 在 $B \geq 4$ 时,

$$N_{\ell-1} \leq 2B \sum_{j=0}^{\ell-2} B^j = \frac{2B}{B-1} (B^{\ell-1} - 1) \leq 3B^{\ell-1}.$$

于是 $N_{\ell-1} \leq x$, 从而 $k \geq \ell - 1$ 。若 $x < N_\ell$, 则 $k = \ell - 1$ 。若 $N_\ell \leq x < N_{\ell+1}$, 则 $k = \ell$ 。若 $N_{\ell+1} \leq x$, 则 $k = \ell'$, 其中 $\ell' > \ell$ 是满足 $n_{\ell'} > 0$ 的最小下标; 这样的下标必然存在, 因为 $x < N$ 。

寻找 ℓ' 等价于寻找最小的 $\ell' > \ell$, 使 $n_{\ell'}^0 > 0$ 或 $n_{\ell'}^1 > 0$ 。可以按如下方式完成:

$$\ell' \leftarrow \left\lfloor \frac{\text{lsb}((N_0 | N_1) \& (\neg M_{\ell+1}))}{b} \right\rfloor,$$

其中 $\text{lsb}(x)$ 是最低有效置位的下标, $|$ 表示按位或, $\neg$ 表示按位取反。

对任意数 x , $x \& (-x)$ 只保留 x 中最右侧的 1。因此, lsb 可以轻松归约到 msb :

$$\text{lsb}(x) = \text{msb}(x \& (-x)).$$

目前并不知道反方向的归约。

求一个数 x 的最高有效置位下标, 等价于求二进制对数的下取整:

$$\text{msb}(x) = \lfloor \lg x \rfloor.$$

Knuth [12] 讨论了若干在常数时间内求一个机器字最高有效置位的方法。若硬件支持, 一种方法是把 x 转换为浮点数并提取指数。Fredman 与 Willard [7] 给出了一种巧妙方法, 只使用常数级逻辑、移位

和算术操作即可求出 $\text{msb}(x)$ ；该方法需要乘法。乘法在这里是必要的。Brodal（见 Knuth [12]）给出了一种不使用乘法、只使用标准逻辑、算术和移位操作的方法，其时间为 $O(\log \log w)$ ，其中 w 是一个机器字的位数。Brodnik 等人 [3] 给出了匹配的下界。

获得与 r 无关的常数访问时间主要具有理论意义，因为实践中使用的 r 预计很小。用于定位 k 的简单 $O(\log r)$ 二分查找，甚至朴素的 $O(r)$ 扫描，通常更适合实际实现。